

Introduction to the Unified Modeling Language (UML)

Part 1 - EX-P1

Marcello Missiroli & Giancarlo Succi



UML OOP Bases

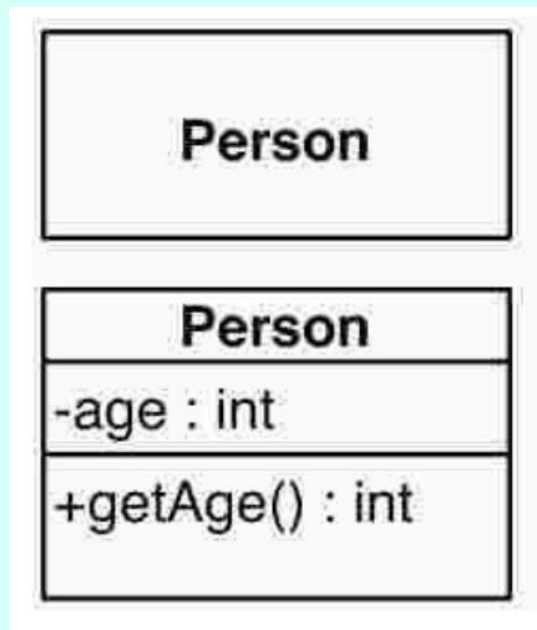


UML Class Icon

- A class has a graphical representation in the form of a rectangle divided into up to three parts or slots.
- – **the name and optional stereotype** (in UpperCamelCase – mandatory)
- – **attributes/properties**
(in lowerCamelCase – optional)
- – **operations/methods**
(in lowerCamelCase – optional)
- The same class may appear with different amounts of decorations in different diagrams.

Classes in UML

Both examples are legal UML





Attributes & Methods

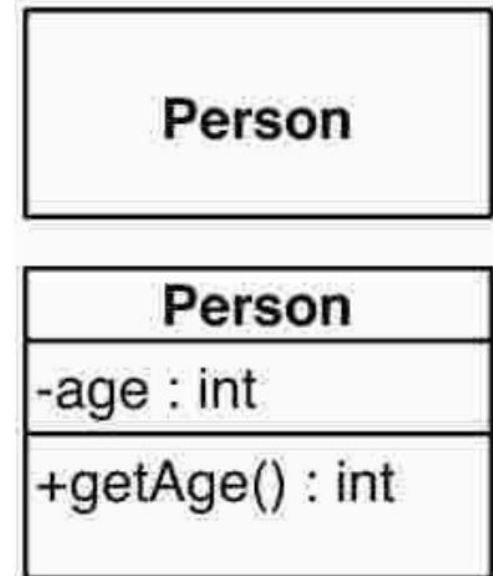
Attribute format:

visibility name multiplicity:Type =
initialValue

Operation format:

visibility name
(parameterName:parameterType, ...) :
returnType

Only the name is mandatory.





Attributes and Operations: Visibility Types

- **+**
Public : Every element that can access the class can also access all of its public members.
- **-**
Private: Only the operations of the class itself can access its private members.
- **#**
Protected: Only the operations belonging to the class or its descendants can access protected members.
- **~**
package: Any element in the same package as the class (or in one of its nested sub-packages) can access the class's package-visible members.



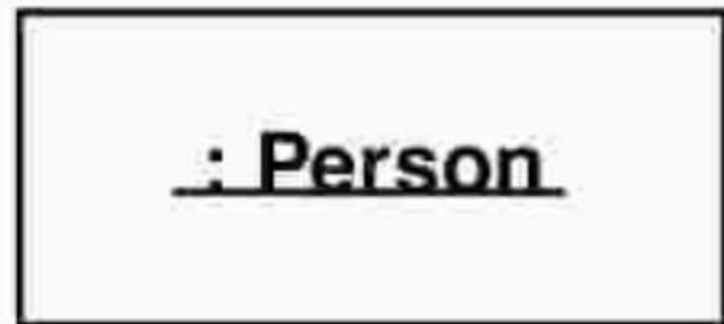
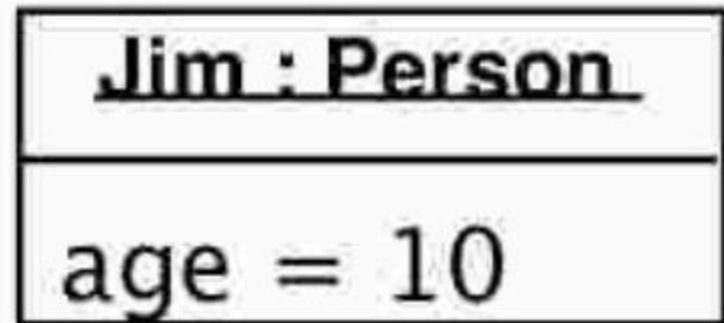
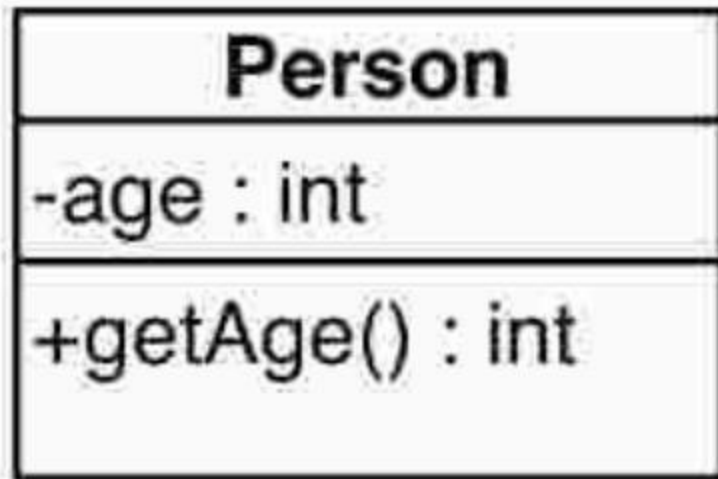
UML Object icon

Instances of classes (objects) have a very similar notation to classes.

The title of an object is **underlined** and follows the format:
name : ClassName
(with the name being optional).

Objects do not have a slot for operations,
but they can define values for their attributes.

UML Object icon





UML Object icon

Instances of classes (objects) have a very similar notation to classes.

The title of an object is **underlined** and follows the format:
name : ClassName
(with the name being optional).

Objects do not have a slot for operations,
but they can define values for their attributes.

Class-object relationship

This is represented as a **dependency** with the stereotype «instanceOf»
(the object “instantiates” the class).

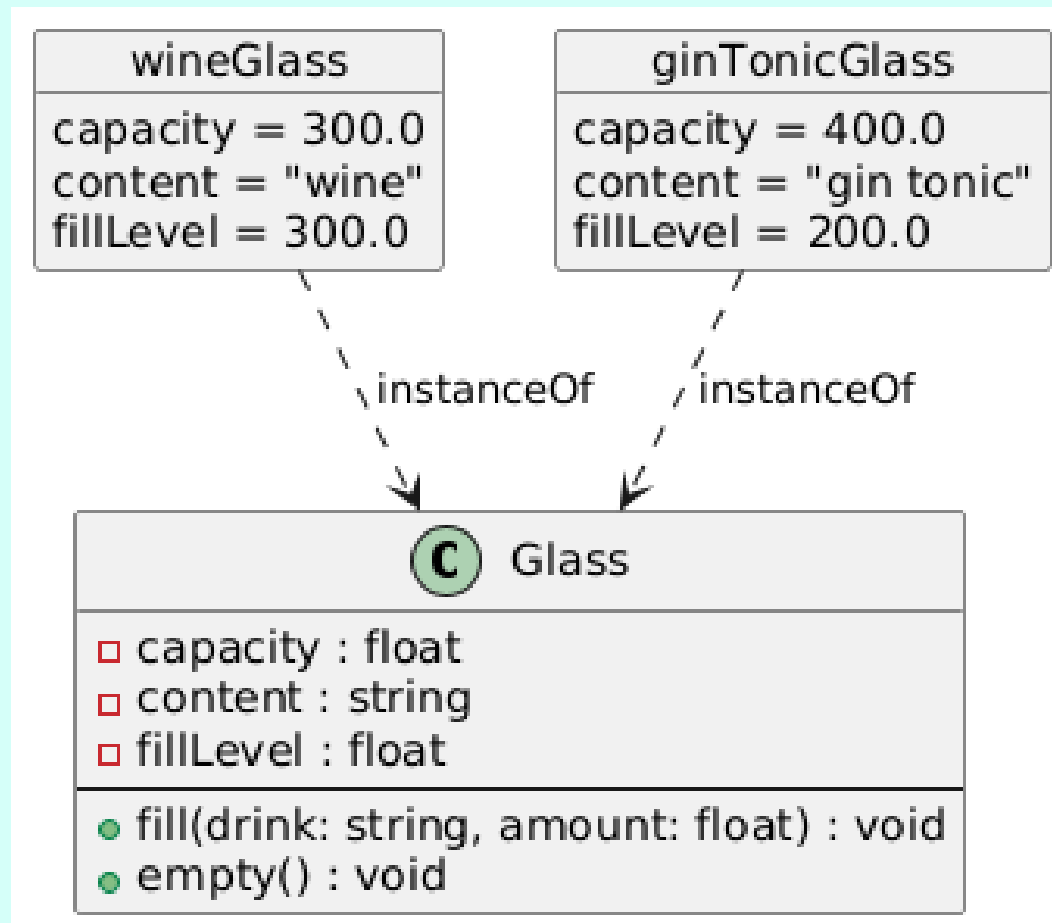




Exercise 1

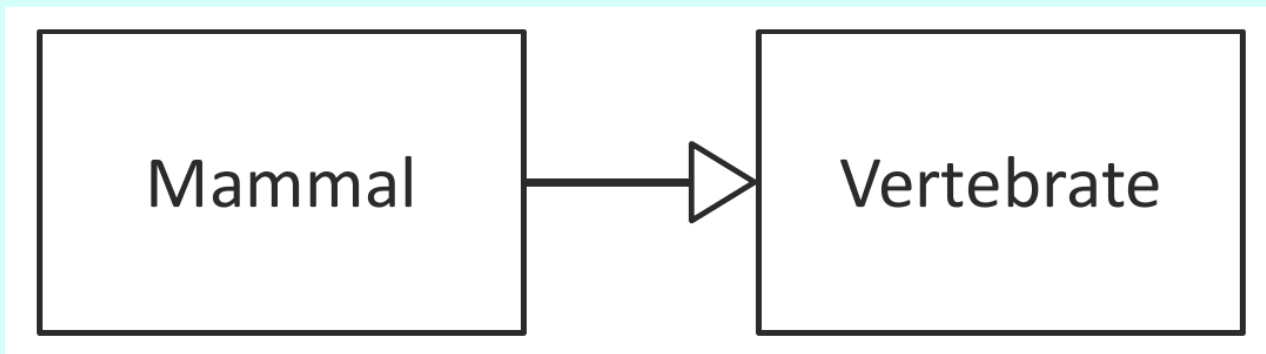
- Represent in UML a class called “Glass”, with at least 3 attributes and 2 methods. Then instantiate 2 glasses, one full of wine, the other half full of gin tonic

Exercise 1 – possible solution



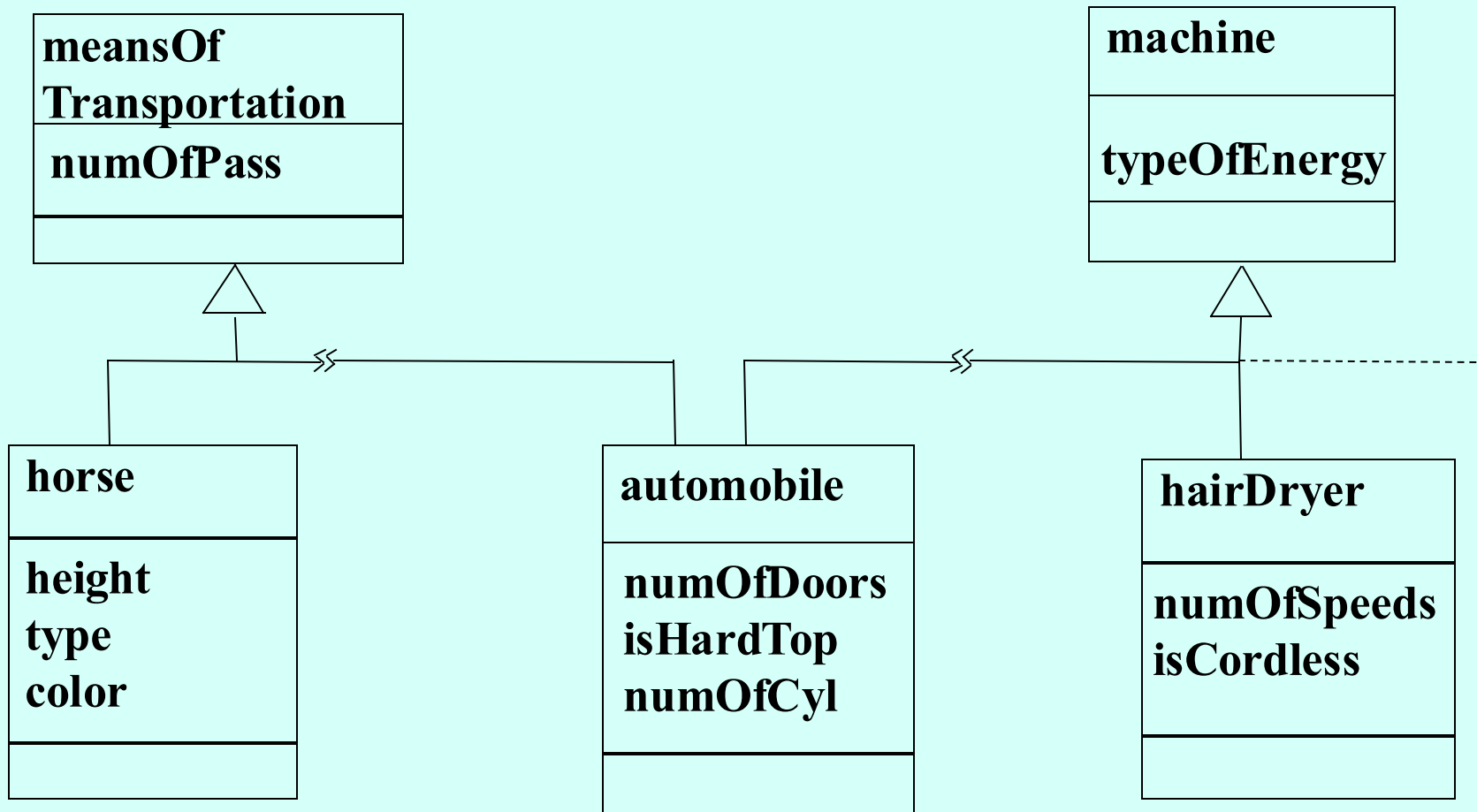
Inheritance in UML

- Inheritance is represented by the **Generalization** relationship in UML.
- The arrow starts from the specific (child) element and points to the more general (parent) element.
- Among all UML relationships, this is the strongest and most constraining.



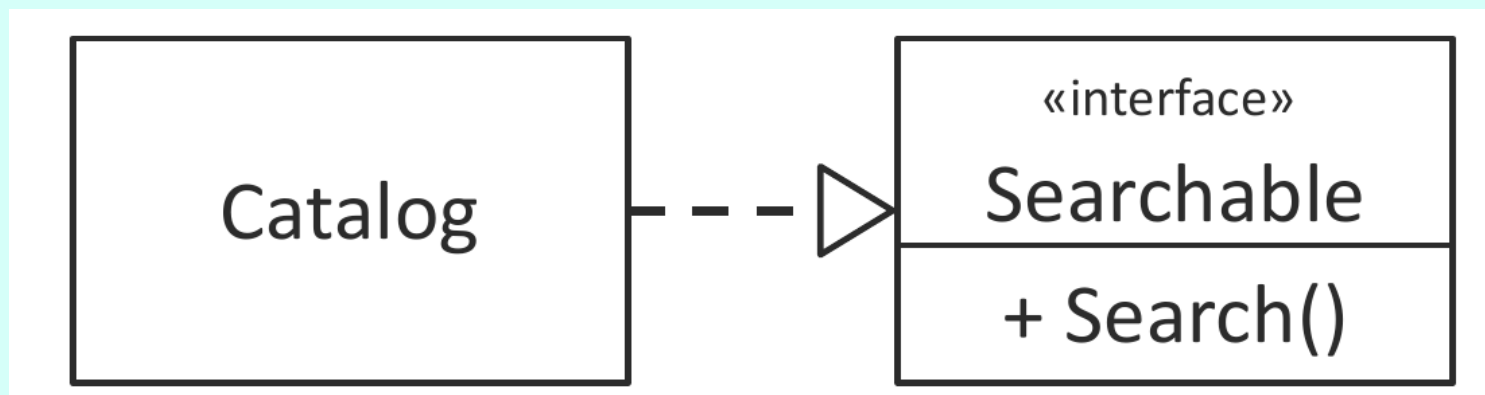


Multiple Inheritance



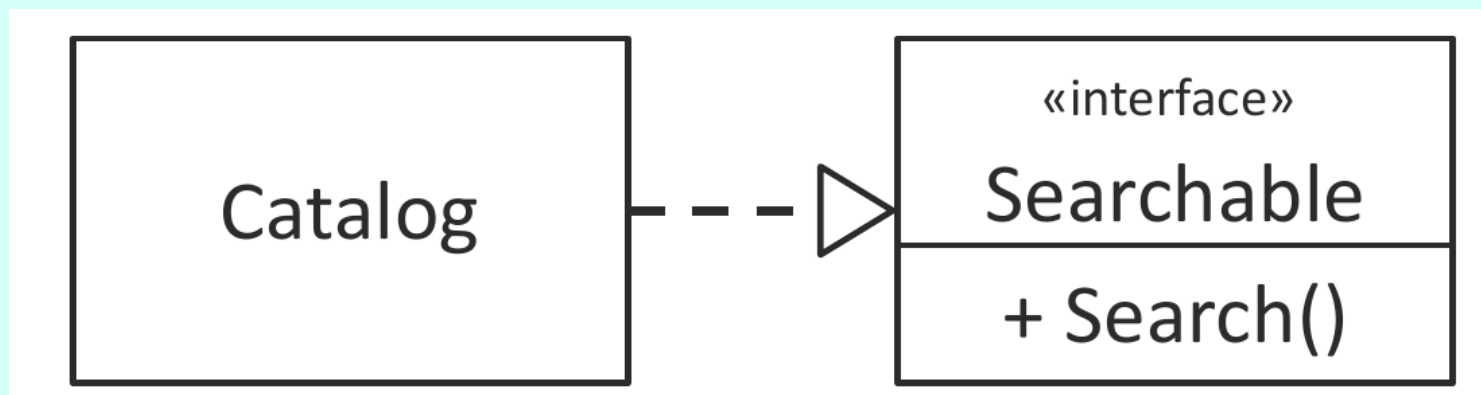
Realization relationship

- A semantic relationship in which the supplier provides a specification, and the client realizes it (by implementing and executing it).
- The canonical example of a realization is when the supplier is a Java-style interface, and the client is the class that implements it.



Realization relationship

- A semantic relationship in which the supplier provides a specification, and the client realizes it (by implementing and executing it).
- The canonical example of a realization is when the supplier is a Java-style interface, and the client is the class that implements it.





Abstract classes and methods

- Abstract classes and methods are written in *italic*
- However, since this is difficult to represent and notice in some cases you can
 - Use the `<<abstract>>`
 - Mark the constructor as protected (`#`)
 - Annotate methods as `<abstract>`

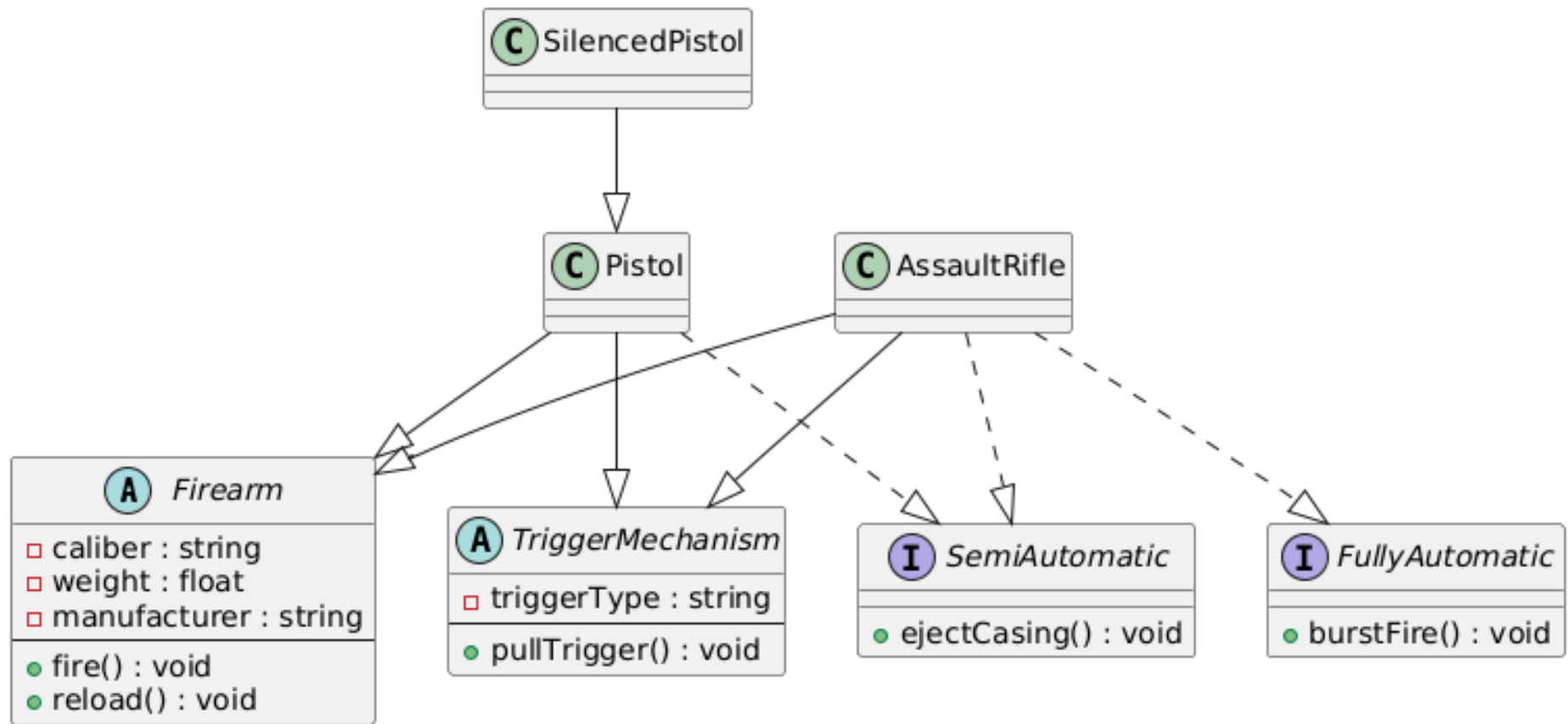


Exercise 2

- Design a UML class diagram for a system that models firearms and their firing mechanisms. Begin with an **abstract class Firearm**, which includes attributes like caliber, weight, and manufacturer, and declares an **abstract method fire()**, along with a **concrete reload()** method.
- Include an **abstract class TriggerMechanism** with a **triggerType** attribute and an **abstract method pullTrigger()**.
- Add **two interfaces**: **SemiAutomatic** with the method **ejectCasing()**, and **FullyAutomatic** with the method **burstFire()**.
- Then, model a **Pistol** that inherits from **Firearm**, uses a trigger mechanism, and implements **SemiAutomatic**. An **AssaultRifle** should also inherit from **Firearm**, use a trigger mechanism, and implement both interfaces. A **SilencedPistol** is a special type of **Pistol**



Exercise 2 - Solution





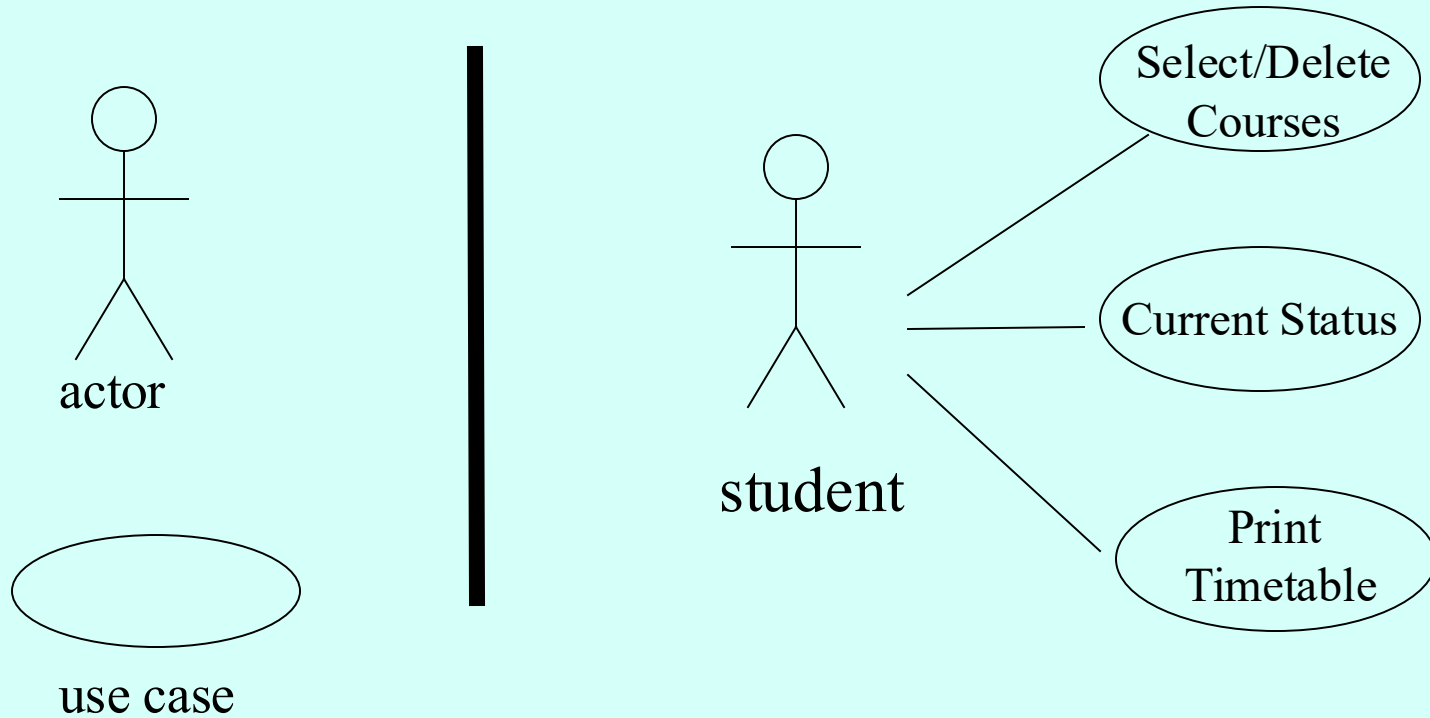
Use Cases Diagram



Use Cases diagram

- A use case diagram expresses a behavior, either desired or offered.
- The subject under examination is usually a system or a part of it.
- It identifies:
 - Who or what interacts with the system (actor)
 - What the actor can do (use case)
- It is typically the first type of diagram created in a development process, as part of requirements analysis.

Use Case Diagram - Early Example



Actors

- An actor specifies a role played by a user or another entity that interacts with the system within the scope of a use case.
- An actor is external to the system.
- It is not necessarily human — it can be a physical object, a software agent, or environmental conditions, etc.
- Actors perform use cases:
- First, identify the actors, then identify their associated use cases.

How to identify actors?

- Who/what uses the system?
- What role does that entity play?
- In which part of the organization is the system used?
- Who/what starts the system?
- Who will support and maintain the system?
- Do other systems interact with this one?
- Are there periodic functions? (e.g., backups)
- Who/what provides or receives information from the system?
- Does an actor have multiple roles?
- Is the same role assigned to multiple actors?

Use case

- A use case is the **specification of a sequence of actions**, including any alternative and/or error sequences, that a system (or subsystem) can execute while interacting with external actors.
- A use case describes something the actor wants the system to do:
 - It is described from the actor's point of view
 - It is triggered by an actor's action
 - The name (label) should be based on a verb or a noun expressing an event.



Use Case: goal

- A use case specifies **what** is expected from a system.
- It **hides how** the system implements that behavior.
- It is a sequence of actions (with possible variants) that produces a result observable by an actor.
- Use cases are used to:
 - Describe requirements (early analysis)
 - Validate the architecture and verify the system



Use Case

- A use case is always initiated by an actor: in UML, an event is always linked to the entity that generated it.
- The actor that initiates the use case is called the primary actor
- Other actors involved during the execution are called secondary actors
- A use case is a classifier with behavior:
 - It can be specified with state diagrams, interaction diagrams, sequence diagrams, and can include preconditions and postconditions.
 - It may also contain variations from the main flow of behavior.

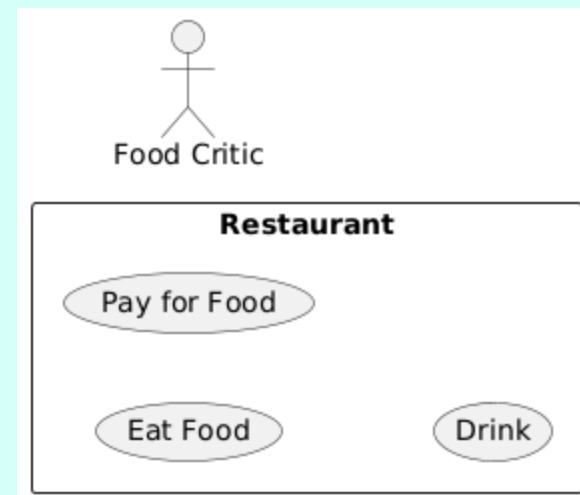


How to identify use cases?

- What functions does each actor expect?
- Does the system manage information (store/provide)?
→ Which actors trigger this?
- Are any actors notified when the system state changes?
- Do actors need to notify the system of sudden changes?
- Do any external events affect the system?
- Which use cases relate to system maintenance?
- Are all functional requirements covered by use cases?

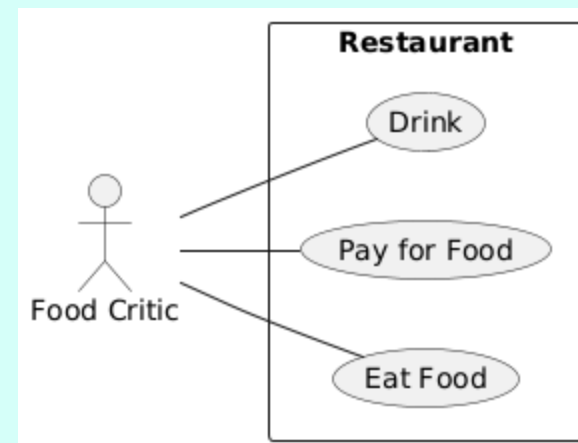
The System

- A simple box enclosing the system
- Actors are always external to the system



Associations

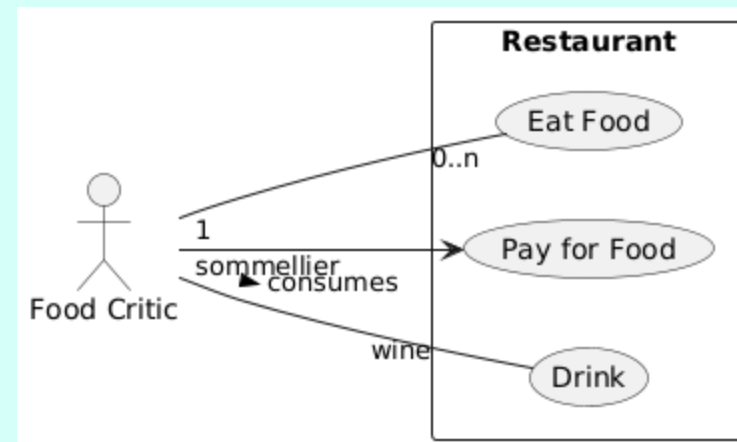
- An association connects actors to use cases.
- An actor can only be associated with use cases, classes, or components (which in turn may be associated with other use cases, classes, and components).
- A use case should not be associated with other use cases that concern the same subject.



Associations

Optional features common to all associations in UML:

- Name
- Multiplicity
- Roles





Easy exercise (with pitfalls)

Consider the definition of a dice game called **Stupid Craps**.

The player places a bet on a number between 1 and 6. The player then rolls a die. If the chosen number appears, the player wins five times the original stake.

Task:

Draw a Minimal UML Use Case Diagram for this system.

You have 3 minutes to complete the exercise.

The work may be done individually or in pairs.

Spoiler?

Easy exercise (solution)

Just one Actor and some UC

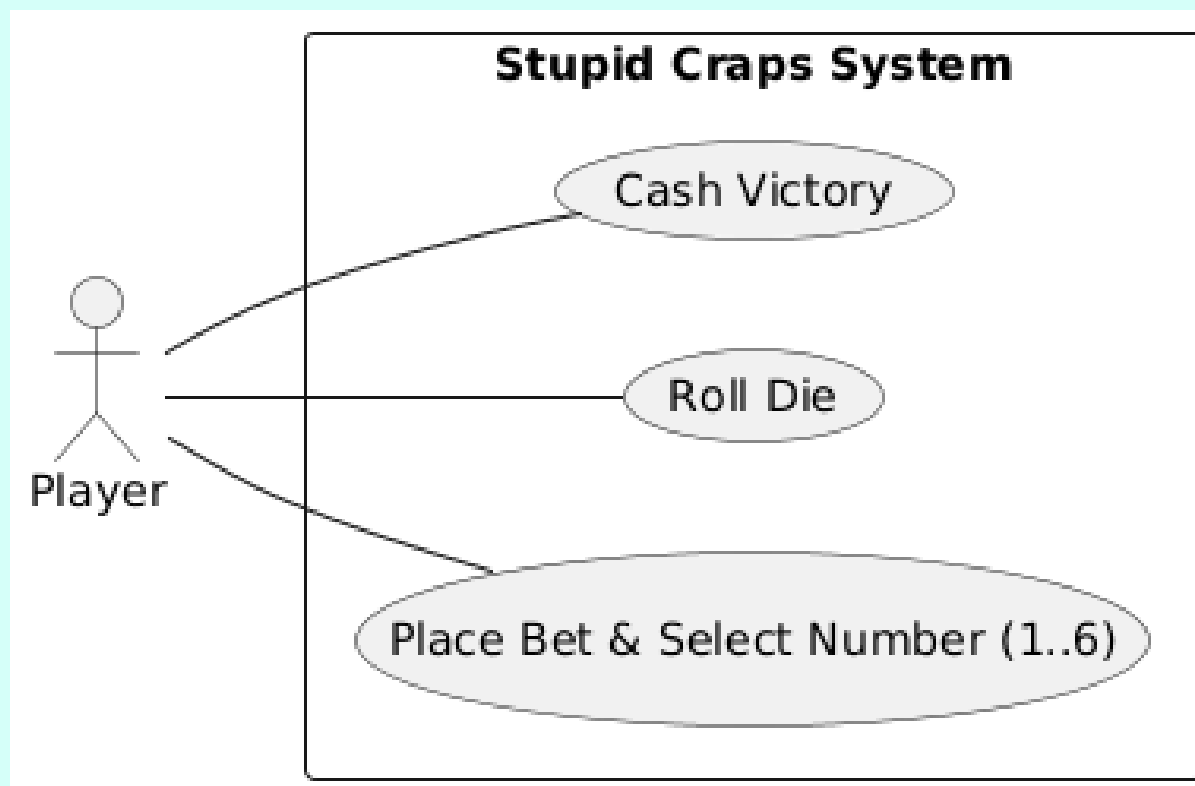
No sequentiality

No dice

No classes

No conditionals

Did you overdo??



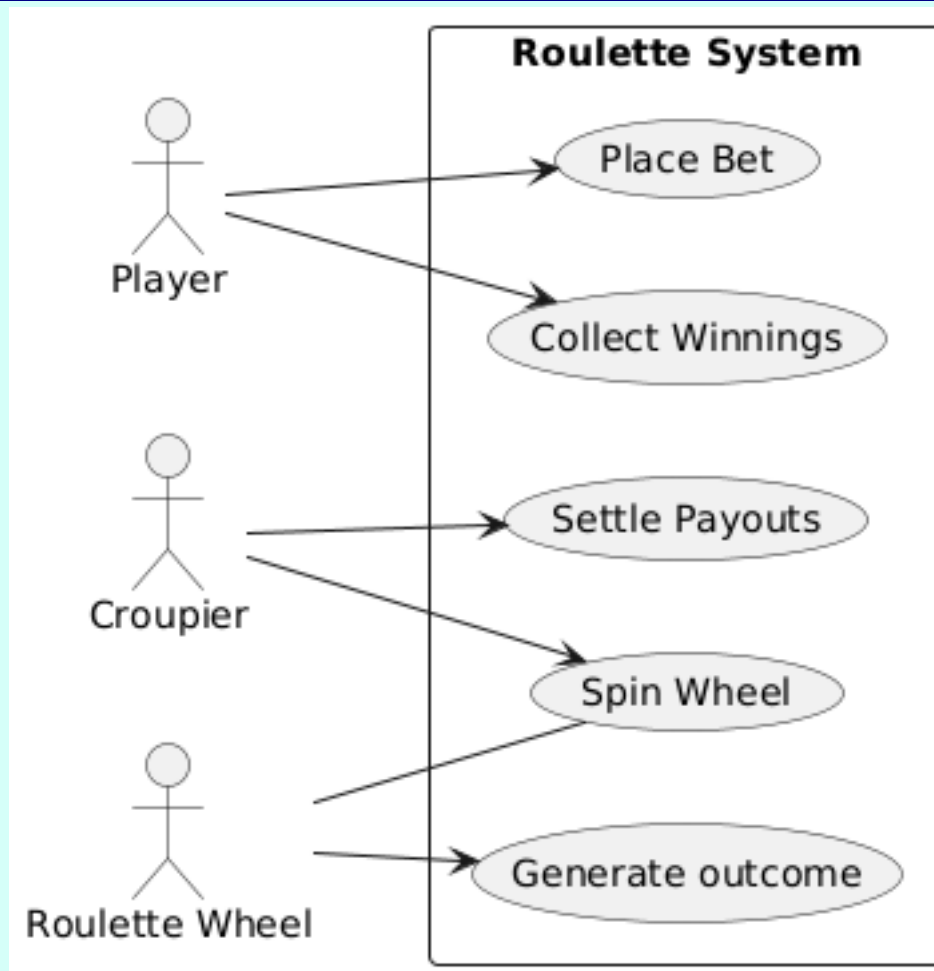


Exercise - Roulette

Design a UML Use Case Diagram for a simplified roulette system.

- The Players places bets, collects winnings.
- The Roulette Wheel is external to the system and provides the spin result (e.g., number and color).
- The Croupier spins the wheel.
- Payouts are simplified:
 - Straight number = $35 \times$ stake
 - Red/Black, Even/Odd, High/Low = $1 \times$ stake

Exercise - Roulette solution



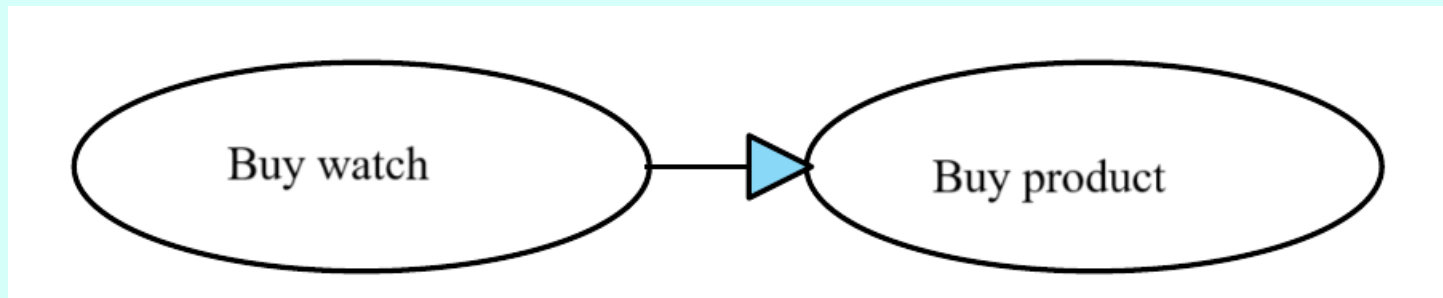


Extends relationship

- Extends: One use case is similar to another but does a bit more
 - Capture the simple, normal use case first
 - For every step ask
 - what could go wrong
 - how might this work out differently
 - Plot every variation as an extension of the use case

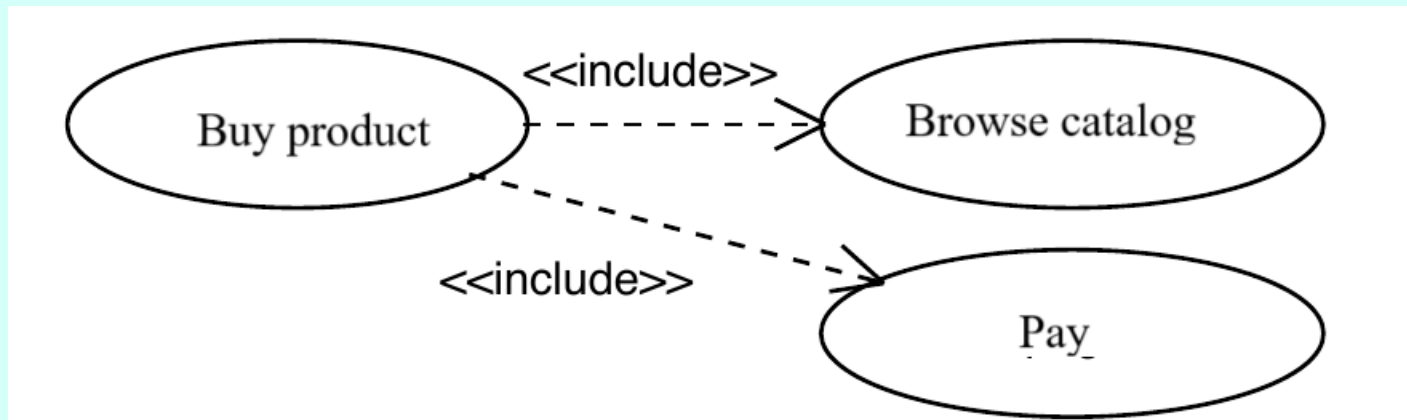
UC elements: generalization

- Akin to Inheritance in OOP
- Connects a Use Case to a more generic one
- The “son” can substitute the “father” wherever it appears



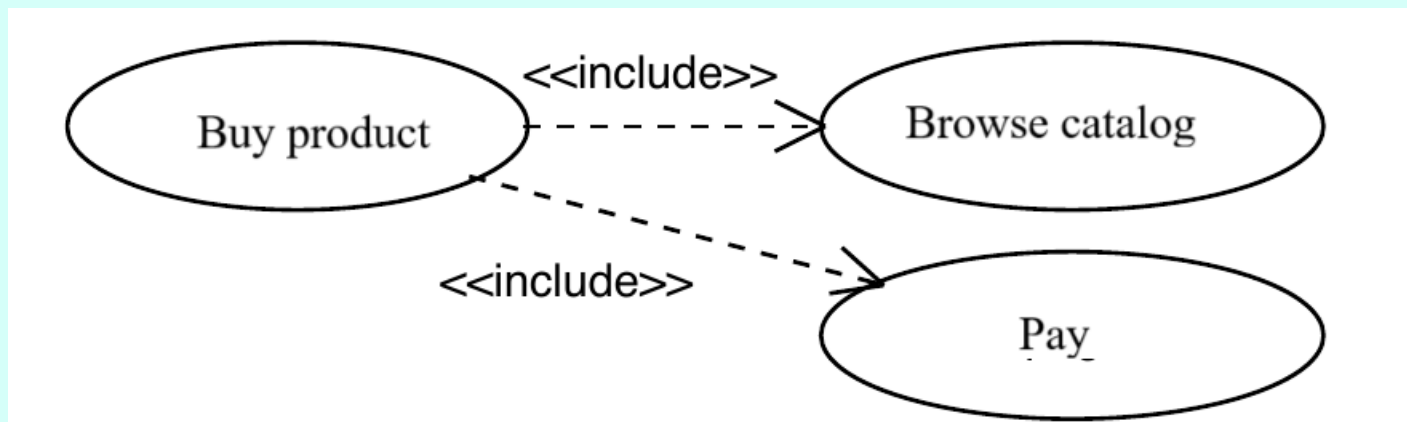
UC elements: include

- A dependency between use cases; the included case is **part of the behavior** of the including one.
- The inclusion is **not optional** and occurs in every instance of the use case.
- The correct execution of the including use case depends on that of the included use case.
- Include cycles cannot be formed.
- It is used to reuse parts common to multiple use cases.



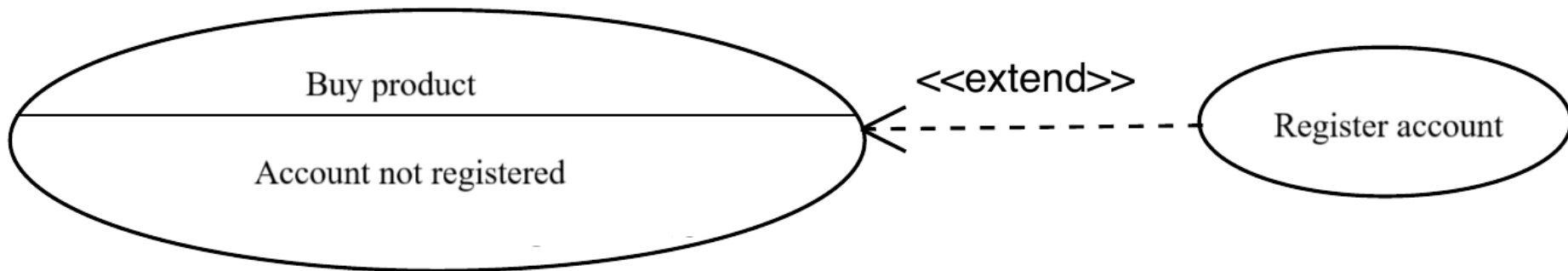
UC elements: extends

- A dependency between use cases (note the direction of the arrow).
- The extending use case (client) specifies an **additional behavior** to the extended one (supplier). This is **supplementary** and optional behavior that handles **special** or **non-standard** cases.
- Different from a generalization between use cases:
 - in a generalization, *both use cases are equally significant*
 - in an extend, the client *does not necessarily make sense* on its own



UC elements: extension point

- A use case reached by at least one extend can **optionally** display its own extension points.
- It specifies **the points and/or conditions** of execution where the behavior is extended.



Comparing extends/includes

Include specifies **mandatory** behavior.

Extend specifies **supplementary** behavior (variants).

- In include, the arrow goes from the including use case toward the included one.
- In extend, the arrow goes from the extending use case toward the extended one.

Both constructs are useful, but they should not be overused, otherwise readability suffers.



UC Elements :Use case description /scenarios

- Generic, step-by-step written description of the interactions between the actor(s) and a use case
- Clear, precise, short descriptions



Textual Description (only for the frequent flyer use case)

The user enters the subsystem to gain more information about its frequent flyer status. Inside the subsystem, the user can access (a) general information about the frequent flier program -the reward schema, how to enroll, how to get miles with partner companies, and (b) specific information on her/his status, such as the miles earned, the status level. The user can also update her/his address.

Solved exercises

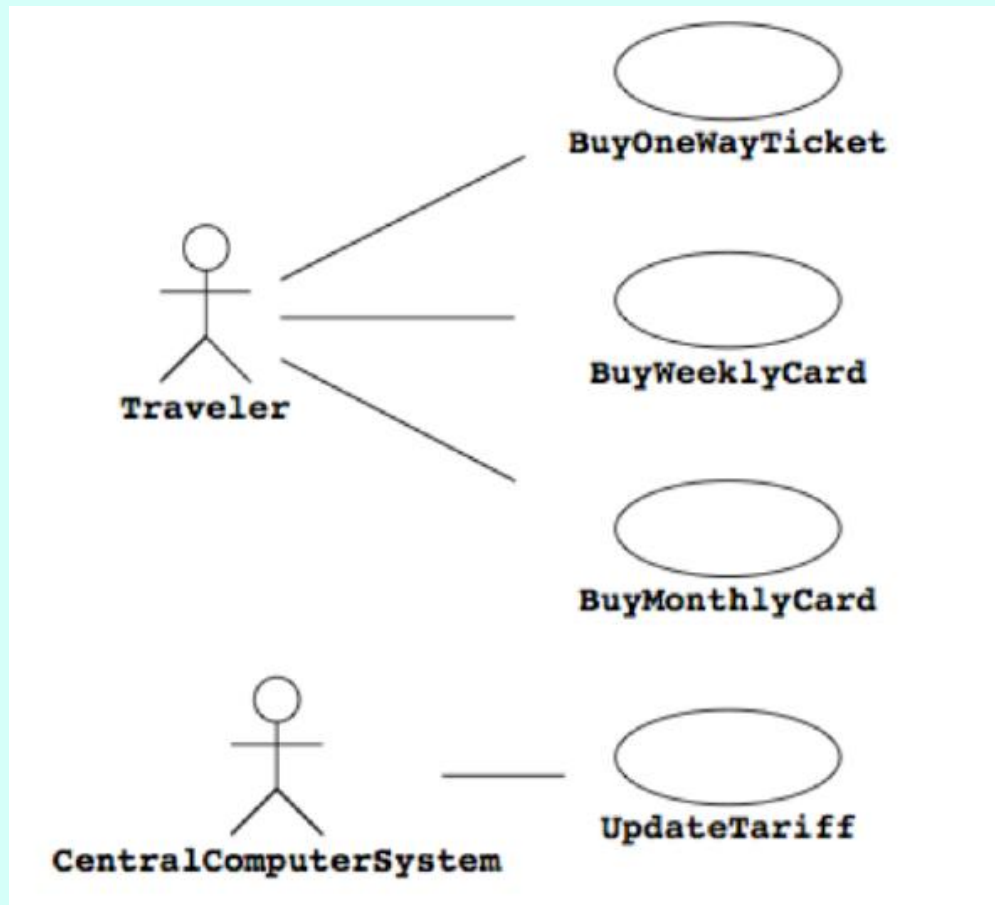
“As always, there is never a 'correct' solution to any modelling problem. It's more that some models are more precise, and more informative, than others. You may find your solutions differ slightly from the ones below, but these solutions demonstrate good practice in model building.”

from “The Models Are the Code - Executable UML”, prof. Paul Krause, University of Surrey

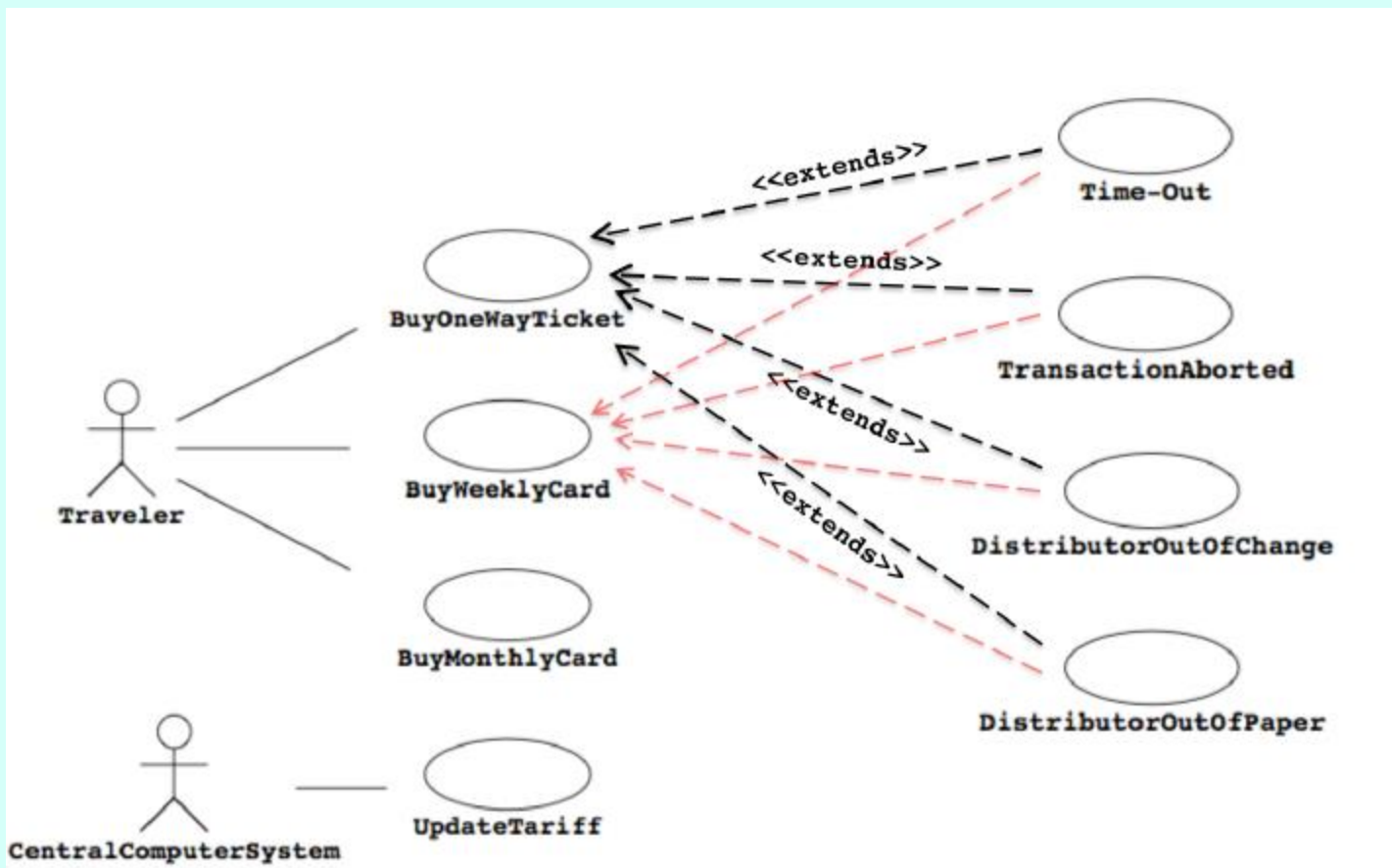
Exercise 1

- Draw a use case diagram for an electronic ticketing system (e.g., railway). The system is used by travelers, who purchase tickets, and by a centralized system, which updates ticket prices.
- Users can make three types of purchases: one-way ticket, weekly pass, monthly pass
- A ticket purchase may fail for four reasons: the buyer took too long to complete the purchase, the buyer explicitly requested not to complete the purchase, the ticket machine ran out of change, the ticket machine ran out of paper

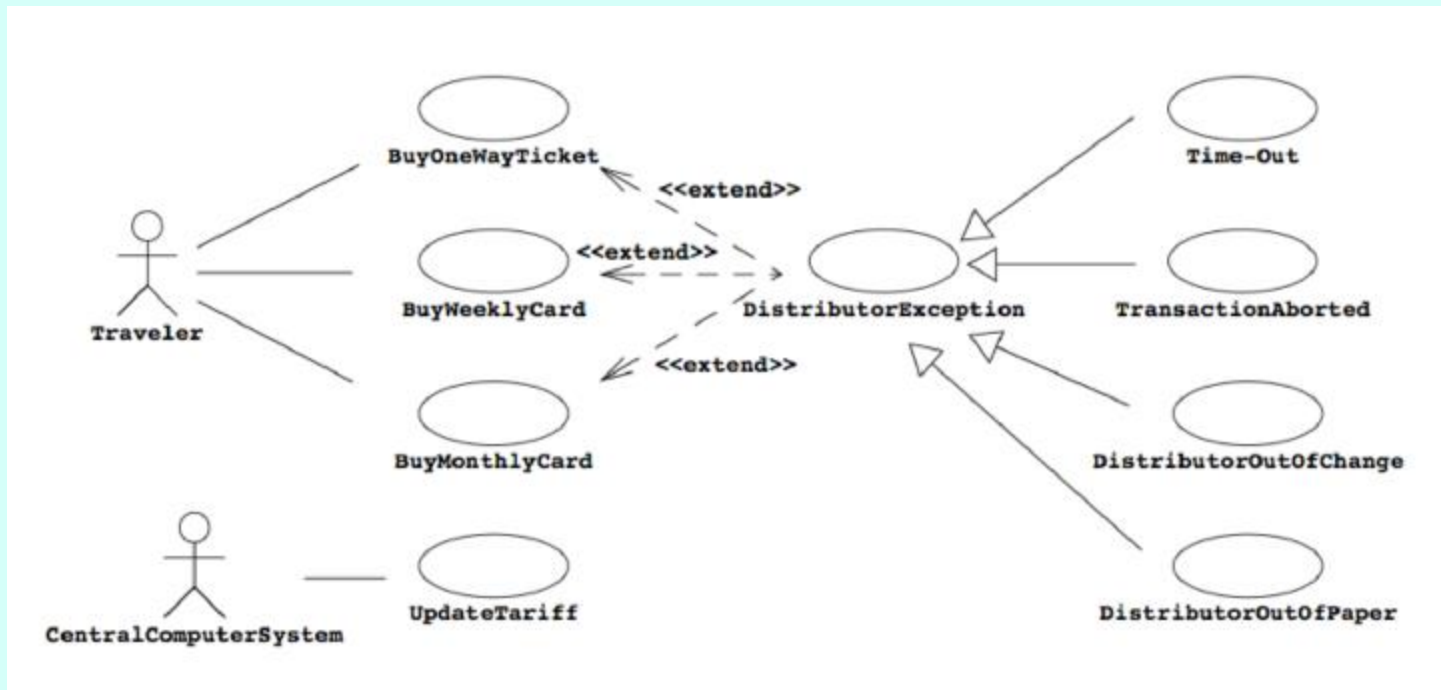
Exercise 1 solution v1



Exercise 1 solution v2



Exercise 1 solution final



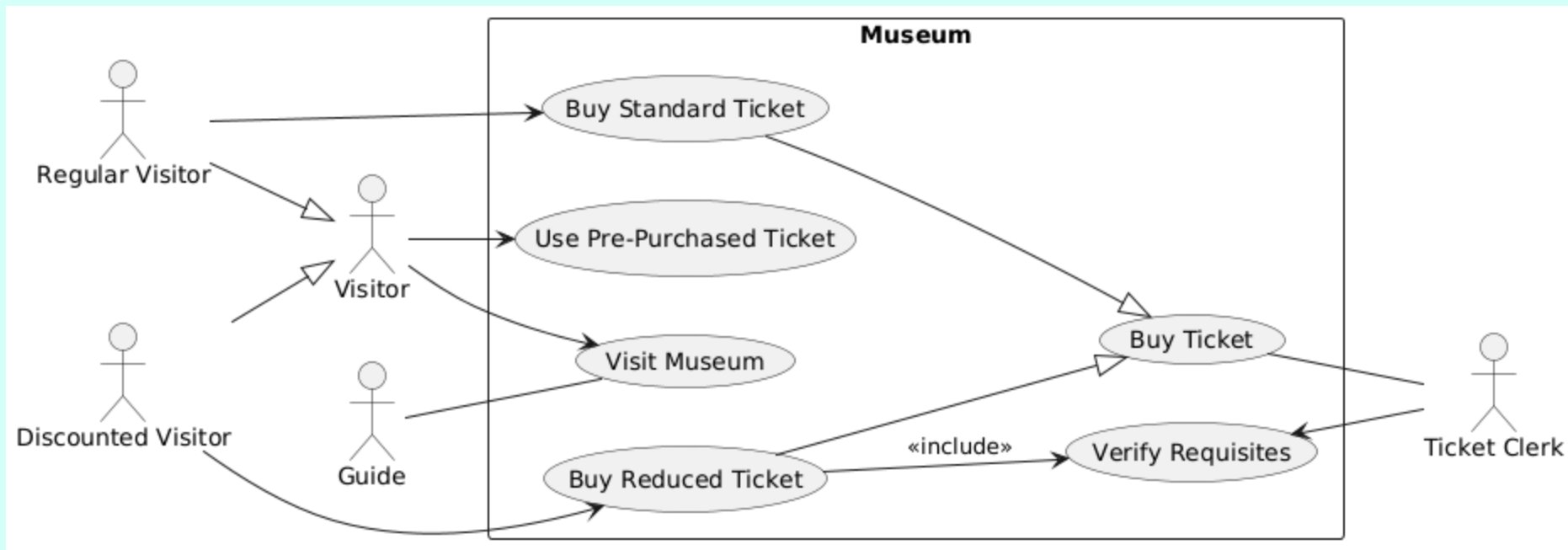
Exercise 2

- Consider a Museum system.
- Users can visit the museum by either buying a ticket sold by a ticket clerk or by using tickets purchased in advance.
- Visits may take place individually or with a guide.
- Some categories of visitors are entitled to a discounted ticket, provided they can prove eligibility for the reduction.
- Represent the system with a use case diagram.

Exercise 2 – Analysis

- Consider a **Museum** system.
- Users can visit the museum by either buying a **ticket** sold by a **ticket clerk** or by using tickets purchased in advance.
- **Visits** may take place individually or with a **guide**.
- Some categories of visitors are entitled to a **discounted ticket**, provided they can prove eligibility for the reduction.
- Represent the system with a use case diagram.

Exercise 2 – Solution



Exercise 4

Consider a store that provides an online catalog freely accessible for consultation.

Registered users can send a purchase order (by providing payment details), which is then stored in the system and forwarded to the order department, which fulfills it.

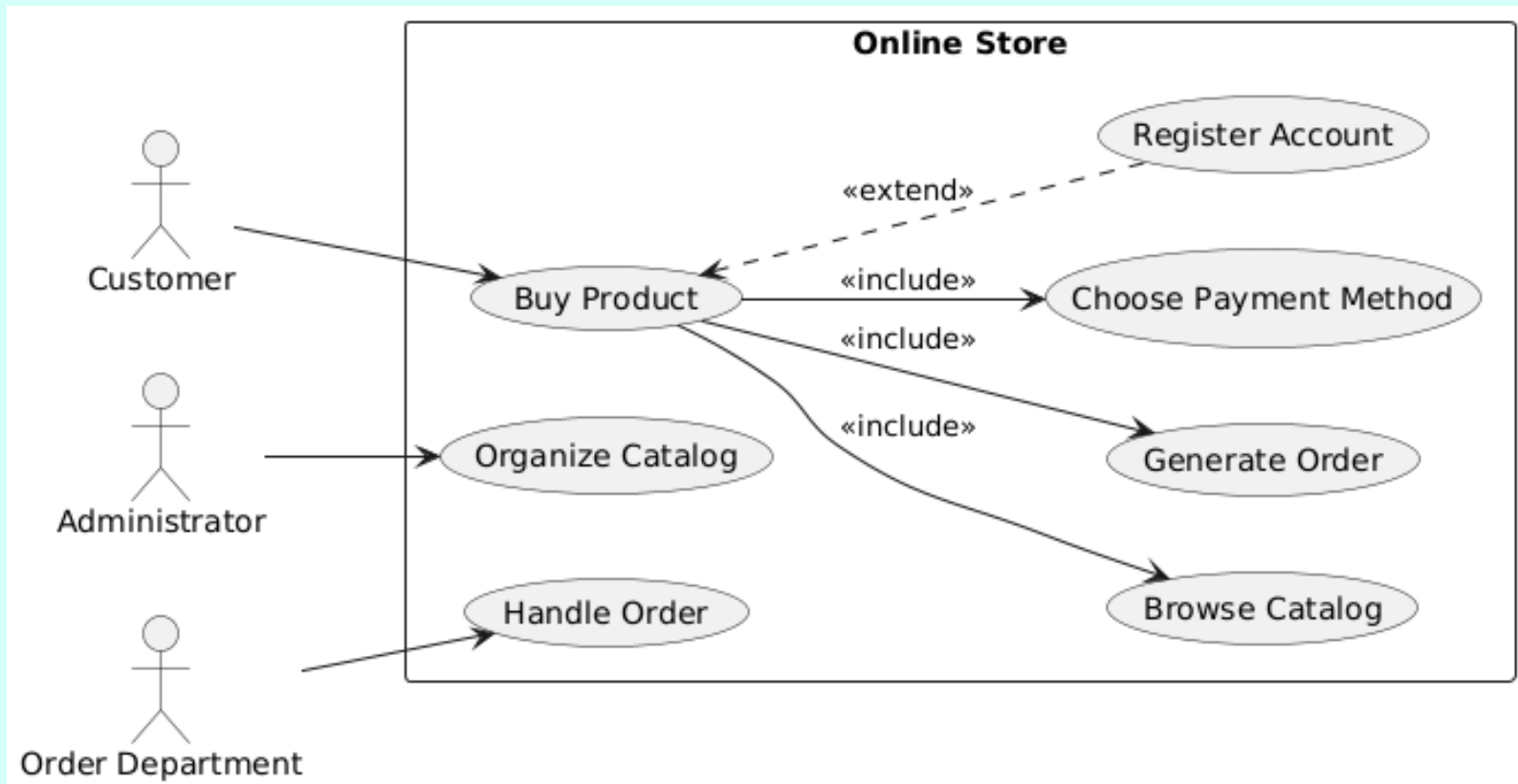
Represent the system with a use case diagram.

- ANALYSIS FIRST: ACTORS? USE CASES?

Exercise 4 - Analysis

- Who interacts with the system (actors)?
 - Customers
 - Online store administrators
 - Order department
- What do they do (use cases)?
 - The customer registers, browses the catalog, and makes purchases
 - The administrator organizes the catalog, which is divided into categories
 - The order department receives orders to fulfill
 - The customer chooses the type of payment

Exercise 4 - Solution





Exercise 4a – Text description

- Provide a text description of the UC “Choose Payment Method”

Exercise 4a – Solution

- When a customer decides to purchase a product, the system must guide them through the payment step. After the order has been generated, the system presents the customer with a set of possible payment methods, such as credit card, PayPal, or bank transfer. The customer selects one of these options and provides the required information — for example, card details in the case of a credit card.
- The system checks that the information is complete and valid. If it is not, the customer is asked to correct the input before continuing. Once the details are properly entered, the chosen payment method becomes part of the order record. At this point the system is ready to proceed with payment processing and order confirmation.
- If the customer abandons the process or cancels at this stage, the purchase is not finalized and the order remains unpaid.



Exercise 4a – Formal Solution

Preconditions:

- The customer has selected one or more products and initiated the Buy Product process.
- An order has been generated and is awaiting payment information.

Main Flow of Events:

- The system prompts the Customer to choose a payment method.
- The Customer selects one option from the available methods (e.g., credit card, PayPal, bank transfer).
- The system collects the required payment details (e.g., card number, expiration date, account credentials).
- The system validates the entered data for format and completeness.
- The chosen payment method is associated with the order.



Proposed Exercise (1)

Stocks Trading Service

- By connecting to the service, a user can connect to different banks to acquire stock prices. The system also allows the user to perform some trend and prediction analysis of prices. If users are interested in ordering some stocks, they can choose to order them immediately or with a delay. They can also either bid at a single price or within a range of prices.
- The system should handle the situation where the connection to a bank is down, there is a conflict of bids, or if a particular stock is no longer available.



Proposed Exercise (2)

Network Printing Service

- There is a super high-resolution colour laser printer available on the network for users to print documents to. The service allows users to preview the output of their document on their screens. In addition, the user can also view the status of the printer to see whether there are other documents waiting to be printed and whether there are any problems with the printer (such as paper jams, out of paper, low on toner, etc...). In addition, users can monitor their own print jobs and delay or delete jobs as they see fit.
- To use this service, a user needs to have the proper authorization and print quota to print. A system administrator manages users and their print quotas.



Proposed Exercise (3)

Component Brokerage System

- This system essentially acts as a broker for software components. When developers have completed development of their software, they can deploy them as reusable software components for others to use. By connecting to the system over the Internet, these developers can submit their components to the system. An administrator then reviews the component for its functionality and ways of connecting to other components, categorizes it, and publishes it in a publicly-viewable area.
- Customers (such as other developers) can then connect to the public system and browse/search the components. When they have found something useful, they can download it for use on their own machine.
- Later, the providers of the components can connect to the system and view the download statistics of their components. They can also add/remove components from the system.



Proposed Exercise (4)

OOCM for *Bug Tracking System*

- Developers use this system to track bugs in an on-going software project. Developers who find bugs can submit a report. Other developers can then assign the bug to a particular developer (especially the developer responsible for the software module) to fix it. In addition, users can browse/search all the bugs in the system so far.
- An administrator manages users to restrict access to the bug tracking system. In addition, the administrator should also be able to generate reports on the state of the project in the form of a set of web pages updated daily at 2am.